

AI와 함께 일하는 새로운 표준

코드를 '빨리' 짜는 시대를 넘어,
'안전하고 일관되게' 설계하는 시대로.

코드를 '짜는' 시대에서 AI에게 '지시하는' 시대로

AI 도구 도입은 이제 선택이 아닙니다. 진짜 문제는 어떻게 제대로 쓰는가입니다.

180만+

GitHub Copilot 사용 개발자 수

76%

AI 코딩 도구 사용/사용 계획

55%

코드 작성 속도 향상 체감

26%↓

PR 병합 소요 시간 단축

BUT — 현장 개발자들의 속마음

"빠르게 만들었는데
나중에 고치기 더 힘들다"

"테스트 코드는
도대체 누가 짜는 건가?"

"팀원마다 AI 코드
스타일이 너무 다르다"

"AI 코드 그대로
믿고 배포해도 될까?"

AI 코드의 불편한 진실: 빠르다고 안전한 것은 아니다

작동은 하는데, 왜 작동하는지 모르는 코드가 조용히 쌓이고 있습니다.

Code Gen



Review



Deploy



Maintain

01

테스트 부재

명시적 요청 없으면 테스트 생략
AI는 통과 위해 값 하드코딩

▲ 검증되지 않은 코드 누적

02

맥락 소실

새 대화마다 컨텍스트 초기화
팀 컨벤션 매번 재입력

▲ 팀 기준과 어긋난 산출물

03

파편화

A·B·C 팀원마다 다른
아키텍처와 스타일 도출

▲ 리뷰 비용 급증

04

결정 부재

"왜 이 구조인가"는
코드에 남지 않음

▲ 블랙박스화

05

운영 불안

타임아웃, 예외, 동시성 등
비기능 요구 미흡

▲ 검토 부담 폭증

'AI를 쓰는 것'과 'AI와 제대로 일하는 것'의 차이

이 간극을 채우고, AI 산출물을 통제 가능한 시스템으로 만드는 구조적 뼈대.

BEFORE

AI에게 코드를 만들게 하는 것

- 빠른 생성
- 파편화
- 불안감
- 블랙박스



Harness
Engineering

AFTER

AI가 만든 코드를 믿고 운영하는 것

- 품질 보장
- 표준화
- 안정성
- 통제력

CHECK POINTS

✓ 표준화 ✓ 품질 구조화 ✓ 최소 구성으로 즉시 적용

소프트웨어 패러다임의 세 번의 전환

Andrej Karpathy의 Software 1.0 → 2.0 → 3.0 프레임.

Software 1.0

Human Code

명시적 로직 설계

개발자가 직접 if-else, 함수,
클래스를 설계하고 통제

LIMIT

복잡한 패턴 인식의
규칙화 한계

Software 2.0

Data + Weights

신경망 학습

데이터와 목적함수를 통해
모델이 가중치 조정

LIMIT

내부 동작 이해 곤란
(Black-box)

Software 3.0

Natural Language

자연어 프롬프트

의도를 자연어로 설명하면
LLM이 즉시 코드 생성

LIMIT

품질·안전·일관성
보장 체계 부재

"코드는 LLM이 만들지만, 안전하게 동작시키는 방법론은 아직 정립 중."

AI 활용 개발의 4단계 진화

자동완성 → 프롬프트 → 컨텍스트 → Harness. 각 단계는 이전 단계의 한계에서 태어난다.

01

AI Autocomplete

↑ 보일러플레이트 작성 속도 향상

↓ 함수 단위 제안, 전체 설계 개입 불가

GitHub Copilot

02

Prompt Engineering

↑ 자연어로 기능 단위 코드 생성

↓ 대화종료시 맥락 소실, 개인 역량 의존

ChatGPT

03

Context Engineering

↑ 팀 규칙·도메인 지식의 자동 적용

↓ 출력 품질 검증 수단 부재

Cursor · AGENTS.md

04

Harness Engineering

↑ 품질 구조적 검증, 설계 결정 보존, 운영 내장

↓ 초기 설정비용 및 팀 학습 곡선

TDD · ADR · CI/CD

진화의 트리거: 마찰이 다음을 낳는다

각 단계의 한계가 다음 단계를 낳는다. 필요에서 태어난 구조.

AI Autocomplete

"한 줄씩은 좋은데, 기능 하나를
통째로 만들 수는 없나?"

Prompt Engineering

"매번 같은 규칙을 설명해야 하나?
팀원마다 결과가 다른데?"

Context Engineering

"규칙을 줬지만, AI가 만든 코드를
어떻게 믿고 배포하지?"



HARNESS ENGINEERING

검증 수단 부재 → **TDD 통합**

AI 출력이 규칙을 지켰는지
자동으로 검증

설계 결정 소실 → **ADR 보존**

"왜 이렇게 결정했는가"
코드 옆에 영구 기록

운영 안정성 부재 → **Observability 내장**

타임아웃, 에러 핸들링
프로덕션 필수 요소 보장

2025년 산업 현황: 속도의 역설

빠르게 만드는 것과 안전하게 운영하는 것은 전혀 다른 문제다.

76

%

전 세계 개발자 AI 코딩 도구 도입률

Stack Overflow 2025

55

%

AI 도구 사용 시 완료 속도 단축

GitHub Research

80

%

일부 기업 코드 중 AI 생성 비율

Industry Report

BUT — 이면에는 그림자가 있다

▲ 코드 중복률 급증

AI 도구 사용과 함께 리팩터링 필요성 동반 상승

▲ 보안·품질 이슈

도입 후 예상치 못한 취약점과 품질 저하 다수

▲ 블랙박스 장애

원인 불명의 운영 장애 유발

"속도가 통제 범위를 넘어섰을 때, 시스템이 필요해진다."

Vibe Coding: 강력한 시작, 그리고 치명적인 함정

프로토타입에서 운영으로. 느낌(Vibe)에서 시스템(Harness)으로.

THE VIBE

"느낌에 완전히 맡기고, 코드가 존재한다는 것조차 잊어버려라. 보이는 것을 말하고 실행하면, 대부분 동작한다."

— Andrej Karpathy (2025)

GOOD FOR

- 빠른 프로토타입
- 개인 사이드 프로젝트
- 해커톤(48h)
- 기술 탐색

THE REALITY

Carlini's Warning (Anthropic, 2025)

AI는 정확성보다 테스트 통과에 최적화되어 있다. 동작하게 만들라고만 하면 가장 빠른 방법으로 값을 하드코딩하며, 이는 결코 일반화되지 않는다.

DANGER FOR

- 팀 협업 리뷰
- 운영 서비스 디버깅
- 정기 유지보수
- 보안·결제 시스템

"Vibe Coding은 지났다. 다음은 Agentic Engineering 이다." — Karpathy"

프롬프트 엔지니어링: 잘 물어보는 기술

좋은 질문이 좋은 답을 만든다. 하지만 매번 새로 질문해야 한다.

TECHNIQUES 주요 기법

Chain of Thought 사고 연쇄

"단계별로 생각해줘" → 논리적 추론 과정 유도

Few-shot Prompting 예시 제공

"이 형식으로 작성해" → 패턴 학습 및 일관성 확보

Role Prompting 역할 부여

"너는 10년 차 시니어야" → 전문가적 시점 도출

LIMITS ⚠ 구조적 한계

01 지속성 소멸

대화 세션 종료 시 맥락 즉시 소멸.
내일 다시 설명해야 함.

02 팀 표준화 실패

개인 프롬프트 작성 역량(암묵지)에 의존.
팀원 간 결과물 편차.

03 지식 자산화 불가

효과적이었던 프롬프트가

"프롬프트 엔지니어링은 개인의 대화 단위에 머무르는 근본적 한계를 갖는다."

컨텍스트 엔지니어링: 환경을 설계하는 기술

매번 묻는 대신, AI가 일할 "환경"을 문서로 미리 세팅한다.

CONTEXT

환경에 담는 핵심 지식

- 코딩 규칙
PEP8 준수, 타입힌트 필수
- 아키텍처
비동기 블로킹 I/O 금지
- 도메인
DeadlineGuard 패턴 사용
- 응답 방식
숨겨진 동시성 리스크 명시

✓ 해결된 것

반복 설명 제거

한 번 설정으로 프로젝트 전체 자동 적용

팀 일관성 확보

모든 팀원이 동일 컨텍스트 공유 (버전 관리)

⚠ 남겨진 문제

검증 수단의 부재

AI가 규칙을 정말 지켰는지 자동 확인 불가.
결국 '사람의 눈'으로 직접 코드를 열어
확인해야 하는 한계 봉착.

Blueprint to System — 4가지 개념 비교

방식이 다른 면 도달하는 품질도 다르다.

비교 기준	Vibe Coding	프롬프트 엔지니어링	컨텍스트 엔지니어링	Harness Engineering
적용 단위	대화 1회 (즉흥적)	질문 단위 (요청별)	프로젝트 단위	조직/팀 단위 (구조적)
지속성	없음 (휘발)	세션 내 한정	프로젝트 지속	영구 (코드베이스 내장)
팀 적용	매우 낮음 (개인 의존)	낮음 (역량 편차)	중간 (파일 공유)	높음 (표준화·자동 검증)
품질 보장	없음 (동작만 확인)	부분적 (요청 의존)	부분적 (규칙만 정의)	구조적 (TDD·CI·ADR)
문서화	없음	없음 (암묵지)	설정 파일 보존	테스트·아키텍처 보존

"Harness Engineering은 AI 코드의 품질을 구조적으로 보장하는 궁극적 단계다."

Harness Engineering: 통제된 파워, 구조화된 품질

AI 코드 생성기가 아니다. AI가 완벽하게 일할 수 있는 "운영체제"다.

OPENAI DEFINITION · 2025

"*Harness Engineering is the practice of building the infrastructure, tooling, and workflows that allow AI agents to work effectively — with quality gates, observability, and the ability to course-correct.*"

→ AI 에이전트가 효과적으로 동작하도록 돕는 인프라·도구·워크플로우 (품질 게이트, 관측성, 방향 수정 포함)

HARNESS (馬具) 말의 힘을 억누르는 것이 아니라, 원하는 방향으로 이끄는 도구.

3 CORE PURPOSES 3대 핵심 목적

01

출력 품질 보장

"동작하는 것 같다"
→ TDD·CI로 동작 검증

02

설계 결정 보존

3개월 뒤에도 알 수 있도록
ADR로 결정 이유 기록

03

운영 안정성 내장

사후 대처가 아닌, 초기부터
타임아웃·에러 관측 구조화

Harness가 Context를 완벽히 넘어서는 이유

"무엇을 알려줄 것인가(Context)"를 넘어, "어떻게 보장할 것인가(Harness)"로.

LIMIT

검증 수단의 부재

문서에 "테스트 작성하라"고 적음.
결국 사람이 눈으로 확인해야 함.

LIMIT

설계 결정의 소실

"어떻게 만들어라"는 규칙은 남지만
"왜 이 구조인가"는 증발.

LIMIT

운영 안정성의 부재

예외 상황, 타임아웃 등 운영 환경의
현실적 문제 통제 불가.

SOLVED

자동화된 품질 게이트

TDD 작성 + CI 파이프라인 자동 실행
통과한 코드만 병합

SOLVED

결정 배경의 영구 보존

ADR 도입, 대안-트레이드오프 선택 이유를
코드베이스와 함께 영구 문서화

SOLVED

안정성·관측성 초기 내장

DeadlineGuard, Prometheus 메트릭을
기능 개발 전 프로젝트 뼈대에 내장

Harness 도입 전·후: 기술 부채 없는 안전한 시작

처음부터 제대로 시작하여 부채를 남기지 않는다.

BEFORE

도입 전

◆ 개인 편차에 의존

팀원별 스타일·에러 처리 방식 다름

◆ 눈덩이처럼 불어나는 기술 부채

"테스트는 나중에"가 장애를 부른다

◆ 사람의 눈으로 하는 품질 통제

All 대량 코드를 일일이 리뷰

◆ "내 PC에서는 되는데요?"

환경 차이로 원인 추적 불가

AFTER

도입 후

◆ 구조적으로 일관된 팀

린트룰로 도구·사람 무관 동일 규칙

◆ 테스트 기반의 안전한 확장

TDD + CI로 기존 기능 보호

◆ 시스템이 통제하는 병합

기준 미달 코드는 병합 원천 차단

◆ 완벽한 재현성 확보

누가, 어디서 실행해도 동일 결과

3 CORE VALUES

Reproducibility 재현성 · 동일 결과 보장

Stability 운영 안정성 · TDD로 사전 검증

Consistency 협업 일관성 · 개인 편차 최소화

Harness 아키텍처: 처음부터 품질을 강제하는 구조

시작부터 AI와 사람의 명확한 역할 분담을 요구하도록 설계.

HUMAN CONTROL

인간의 제어 영역

통제 및 규칙

`.cursorrules`

AI 행동 규칙서

`/docs/adr/`

설계 결정 기록

사람이 작성하여

AI 품질 경계를 설정

AI EXECUTION

AI의 실행 영역

비즈니스 로직

`/src/core/`

도메인 규칙·체인

`/src/adapters/`

외부 시스템 연동

설정된 규칙 안에서

AI가 100% 코드 생성

SYSTEM GATE

시스템 검증 영역

보호막

`tests/`

TDD 회귀 보증

`Makefile`

단일 진입점

모든 팀원이 동일

환경 명령으로 통일 검증

Harness 프로젝트: 실제 디렉토리 구조

파일 배치가 곧 품질 경계다. 역할이 섞이지 않도록 물리적으로 분리한다.

```

project layout

my-service/
├── .cursorrules           # AI 행동 규칙
├── Makefile              # 단일 진입점
├── pyproject.toml        # 의존성, 린트, 포맷
├── docs/
│   └── adr/              # 설계 결정 기록
│       ├── 0001-history-storage.md
│       └── 0002-timeout-policy.md
├── src/
│   ├── core/             # 순수 도메인 (I/O 금지)
│   │   ├── models.py
│   │   └── rules.py
│   ├── adapters/         # 외부 시스템 연동
│   │   ├── http_client.py
│   │   └── metrics.py    # Prometheus
└── tests/

```

인간 제어

`.cursorrules` / `docs/adr/`

품질 경계 설정

AI 실행

`src/core/` · `src/adapters/`

규칙 안에서 100% 생성

시스템 검증

`tests/` · `Makefile`

"core/는 I/O를 모른다 · adapters/는 도메인을 모른다 · tests/는 항상 먼저 존재한다."

AGENTS.md: 팀 전체의 AI '규약'

파일 하나로 팀 개발 표준을 통합하고, 모든 AI 출력물의 품질을 통제한다.

구분	X 미적용 규약이 없을 때	✓ 적용 규약이 있을 때
AI 출력 결과	타입, 예외, 문서화 누락된 파편화된 최소 코드	타입·예외·Docstring까지 포함된 완성형 코드
팀 워크플로우	팀원마다 프롬프트 달라 품질 편차 발생	누가 요청하든 매번 동일 기준 자동 보장

SETUP · 도입 방법

Step 1

루트에 .cursorrules 생성



Step 2

AGENTS.md 내용 복사·붙여넣기



Step 3

재시작 → 즉시 자동 적용

.cursorrules 실전 예시

파일 하나로 팀 AI 출력을 균일화한다. 복사 → 붙여넣기 → 재시작. 30분 이내 완료.

```

●●● .cursorrules

# Project Rules - Order Management API
# Python 3.11 FastAPI + PostgreSQL

## 언어 스타일
- Python 3.11+, type hint 전 함수 필수
- ruff + mypy --strict 통과 필수
- 함수 20줄 초과 시 분할

## 아키텍처
- async 함수에서 blocking I/O 금지
  (time.sleep, requests, 동기 DB 드라이버)
- 외부 HTTP는 httpx.AsyncClient + timeout=5.0
- 비즈니스=src/core/, 외부=src/adapters/

## 검증
- 신규 기능은 RED 테스트 먼저 (pytest)
- 커버리지 80% 미만 시 병합 차단

## 출력 형식
- 변경 시 의도·트레이드오프·리스크 요약
- 추정 기반 코드엔 # ASSUMPTION: 주석

```

IMPACT 이 파일 하나의 효과

01 반복 설명 제거
프로젝트 컨벤션을 매번 쓸 필요 없음

02 팀원 간 편차 제거
누가 요청하든 동일 기준 적용

03 버전 관리 가능
git diff로 규칙 변경 이력 추적

04 AI 도구 종립
Cursor · Copilot · Claude Code 호환

TDD, AI 시대에 더 중요해진 이유

테스트 없는 AI 코드 = 시한폭탄. 코드가 빠를수록 검증은 더 촘촘해야 한다.

⚠ 사람 vs AI의 코딩

HUMAN 사람

작성 중 로직 이해·검증 완료 → 오류 추적 용이

AI AI

'동작하는 것처럼 보이는' 코드 생성

→ 직접 검증하지 않으면 알 수 없음

TDD 3단계 사이클

RED 실패 테스트 먼저 작성

구현 전 무엇을 만들 것인가 명확히 정의

GREEN 최소 구현으로 통과

AI 프롬프트의 기준점 (테스트가 명세)

REFACTOR 구조 개선

동작 유지, 구조만 변경 (Tidy First)

→ 코드가 곧 명세서. 과감한 리팩토링이 가능해진다.

RED 테스트 실전: AI에게 '무엇을 만들지'를 테스트로 말한다

프롬프트보다 테스트가 구체적이다. 테스트가 곧 명세다.

RED - 먼저 실패하는 테스트

```
# tests/test_order_service.py
import pytest, time
from src.core.order import OrderService

class TestOrderCancel:

    def test_cancel_shipped_raises(self):
        """배송 완료된 주문은 취소 불가"""
        svc = OrderService()
        order = svc.create(items=["item-1"])
        svc.mark_shipped(order.id)
        with pytest.raises(ValueError):
            svc.cancel(order.id)

    def test_cancel_within_5s(self):
        """취소는 5초 내 응답"""
        svc = OrderService(timeout=5.0)
        order = svc.create(items=["item-1"])
        start = time.monotonic()
        svc.cancel(order.id)
        assert time.monotonic() - start < 5.0
```

AI PROMPT

"위 2개의 테스트를 통과시키는
OrderService.cancel()을 구현해줘.
엡지케이스 하드코딩 금지."

WARNING

"if order_id == 특정값: raise"
패턴 보이면 즉시 거부

VERIFIED

- PENDING 주문 취소 → 성공
- 존재하지 않는 ID → NotFoundError
- 이중 취소 → 중복 발생 처리

ADR: 실패와 타협의 기록

코드보다 빨리 잊히는 것은 '결정의 이유(Why)'다. ADR은 이를 코드 옆에 영구 보존한다.

CONTEXT DECAY

결정 맥락의 소실 경로

- ▼ 구두 전달
회발성 정보
- ▼ 슬랙에 묻힘
검색 비효율
- ▼ 담당자 퇴사
완전 소실

? 이 코드 누가 왜 이렇게 짰어?

ADR-0001

히스토리 저장소 선택

상태	Accepted
컨텍스트	DB vs 메모리 결정 필요
결정	collections.deque 사용
대안	MariaDB, SQLite
이유	데모 안정성 최우선, DB 오류 차단

BENEFITS

- ✓ 재논쟁 비용 절감
- ✓ 온보딩 가속
- ✓ AI 프롬프팅 품질 ↑

ADR 실전: 3개월 뒤의 나와 AI를 위한 기록

코드는 "어떻게"만 말한다. "왜"는 ADR에만 남는다.

docs/adr/0002-timeout-policy.md

ADR-0002: HTTP 클라이언트 타임아웃 정책

상태 (Status)

Accepted - 2026-03-14

컨텍스트 (Context)

외부 결제 API 호출 무한 대기로
요청 스레드 누적 → 서비스 전체 장애.

결정 (Decision)

모든 외부 HTTP는 `httpx.AsyncClient`
+ `timeout=5.0초` (`connect=2`, `read=5`)
`DeadlineGuard` 데코레이터로 통합 적용

대안 (Alternatives)

- 무제한 대기: 안정성 ↓, 기각
- 1초: 정상 요청도 실패, 기각

선택 이유 (Reason)

결제 API SLA P99=3초, 5초는 2배 여유.

IMPACT ADR이 해결하는 문제

재논쟁 비용 제거

"왜 5초로 했지?"가 슬랙에서 반복되지 않음

온보딩 가속

신규 팀원이 ADR만 읽으면 설계 이유 파악

AI 프롬프팅 품질

"@ADR-0002 참고해서 retry 추가해줘"

Observability: 실패를 숨기지 않는다

운영에서 '몰랐다'는 말이 나오지 않으려면, 기능 개발 후가 아닌 처음부터 관측 가능하도록 설계.

ANTI-PATTERN

기능 개발



장애 발생



뒤늦게 로그 추가



다시 배포

결과: 초기 장애 원인 추적 불가

HARNESS WAY

Non-blocking 관측성 파이프라인

메인 스레드

요청 수신



로직 실행



emit(event)

즉시 반환



이벤트 큐

Drop 정책

drop_oldest

drop_newest



워커 스레드

비동기 처리

로그·메트릭

Prometheus

필수 메트릭 (Prometheus)

dropped_events · queue_size · worker_alive

관측 큐 Drop 정책: 도메인 SLA가 답을 정한다

"버릴 수 없다"는 환상이 큐 포화를 부른다. 무엇을 버릴지 먼저 정한다.

SCENARIO 워커가 느려져 큐가 포화됨. 새 이벤트를 받을 때 무엇을 버릴 것인가?

drop_oldest

최신성 우선

[구형] →  [새 이벤트] 입장

적합 도메인

- ✓ 실시간 스트리밍 메트릭 (CPU, RPS)
- ✓ 대시보드용 게이지 값
- ✓ 시세·환율·주가 톱
- ✓ 사용자 현재 위치, 재고 수량

drop_newest

기존 보존

[기존 큐 유지]  ← [새 이벤트]

적합 도메인

- ✓ 과금·결제 이벤트 (손실 = 매출 손실)
- ✓ 감사 로그·보안 이벤트
- ✓ 주문 생성, 계좌 이체 트랜잭션
- ✓ 알림 발송 큐 (중복 < 누락)

CI 파이프라인: '내 컴퓨터에선 되는데'의 끝

단순 자동화가 아니다. AI 코드의 품질을 사람 리뷰 전에 구조적으로 걸러내는 게이트다.

Step 01

AI 코드 생성

빠르지만 환경 의존적
오류 발생 가능성 존재

Step 02

멀티 버전 CI 품질 게이트

Python 3.10-3.11 필수 통과
3.12 실험 게이트

Step 03

사람의 리뷰

기계적 오류 제거 완료
설계 적합성에만 집중

ENVIRONMENT CONSISTENCY · 일관성과 재현성 보장

로컬

make test

=

CI 파이프라인

make test

결과: AI 코드 + CI 품질 게이트 + 사람 설계 리뷰 = 흔들리지 않는 품질

Makefile & CI: "내 PC에선 되는데"를 시스템적으로 제거

로컬 명령과 CI 파이프라인이 같은 진입점을 공유해야 한다.

Makefile

```
# Makefile - 단일 진입점
.PHONY: install test lint run observe clean

install:
    python -m pip install -U pip
    pip install -e .[dev]

test:
    pytest --cov=src --cov-fail-under=80 \
        --cov-report=term-missing

lint:
    ruff check src tests
    mypy --strict src

run:
    uvicorn src.main:app --reload

observe:
    docker compose up -d prometheus grafana

ci: install lint test # CI와 동일
```

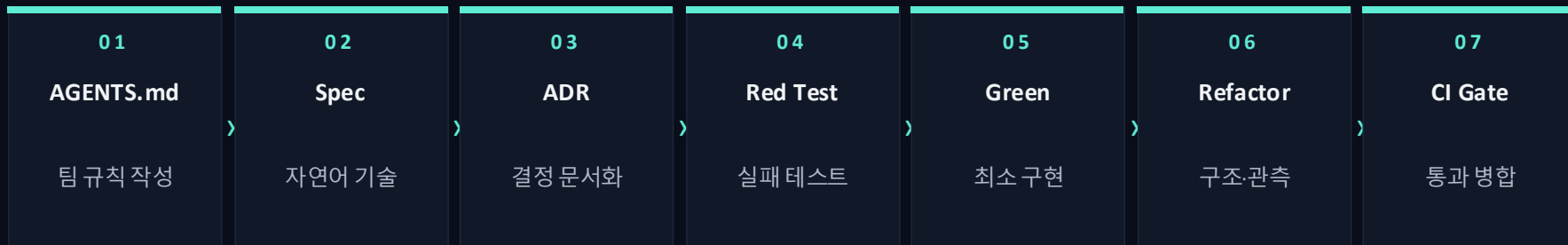
.github/workflows/ci.yml

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]

jobs:
  test:
    strategy:
      matrix:
        python: ["3.10", "3.11"] # 필수
        include:
          - python: "3.12" # 실험
            experimental: true
    continue-on-error: ${ matrix.experimental }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: make ci # 로컬과 동일 명령
```

전체 개발 파이프라인: Spec에서 운영까지 하나의 흐름

개별 도구의 모음이 아닙니다. 매끄럽게 연결된 하나의 파이프라인입니다.



BEFORE 기존의 불안감 → **AFTER** 구조로 해소된 안도감

"이 코드 바뀌도 되나?" ▼ 테스트 돌려보면 안다 TDD	"이거 왜 이렇게 만들었지?" ▼ ADR을 보면 안다 ADR	"내 PC에선 됐는데..." ▼ 환경 차이가 없다 Makefile	"이 에러 왜 났지?" ▼ 메트릭을 보면 안다 Observability
--	---	--	---

최소 구성으로 시작하는 방법: 전부가 아니어도 됩니다

완벽한 구성을 갖추려다 시작조차 못하는 것보다, 파일 1개로 바로 시작.

LEVEL 1

즉시 시작

30분

ADD

.cursorrules

IMPACT

팀 AI 규칙 통일

LEVEL 2

1주일 내

신규 기능에

ADD

ADR 양식·흐름도

IMPACT

설계 결정 기록 습관

LEVEL 3

한 달 내

프로젝트 전체

ADD

Makefile · tox

IMPACT

테스트 표준화

LEVEL 4

완전 구성

3개월, 팀전 확장

ADD

관측성·CI·완전 TDD

IMPACT

Harness 전체 완성

PRINCIPLES

순서 무관 가장 고통스러운 부분부터 · 기존 코드 유지 신규 기능에만 먼저 · 점진적 접근 전부 바꾸려 하지 말 것

극단적 자동화의 한계와 발견

Nicholas Carlini · C Compiler Experiment (Anthropic, 2025)

16	병렬 에이전트 Claude Opus	0	개발자 작성 코드 lines	100K	생성된 컴파일러 코드 Rust 기반
----	------------------------	---	--------------------	------	------------------------

FINDINGS Problem → Solution 매트릭스

▲ 컨텍스트 오염 <i>Context Pollution</i>	→ ✓ 최소화된 테스트 출력 로그 요약 분리
▲ 무한 루프 정체 <i>Time Blindness</i>	→ ✓ Fast Mode 도입 테스트 1~10% 샘플링
▲ 에이전트 뒹어쓰기 <i>Collisions</i>	→ ✓ GCC 오라클 레퍼런스 기준점 분리

⚠ 10만 라인의 상한선: 100% 자율화보다 AI 코드를 검증하는 구조가 현실적 목표.

AI 협업 3대 안티패턴: Carlini 실험이 남긴 교훈

10만 라인을 살아남기 위해 했던 것은, 처음부터 했어야 할 것이다.

01

테스트 통과만 요구

"이 테스트 통과시켜줘"

증상

- 하드코딩
- 엡지케이스만 임시 처리
- 일반화 실패

HARNESS 원칙

테스트는 일반 계약이다.
반드시 2~3개의 변형 케이스 추가

02

전체 컨텍스트 단번 주입

"모든 파일 보고 수정해줘"

증상

- Context Pollution
- 정확도 저하
- 불필요한 수정 유발

HARNESS 원칙

필요한 최소 범위만 전달.
ADR·tests/만으로 의도 전달

03

검증 없는 병렬 에이전트

"여러 작업 동시 진행"

증상

- Collisions
- 상호 덮어쓰기
- 비결정적 결과

HARNESS 원칙

각 에이전트별 분기·PR 분리.
CI가 최종 충돌 검증

통제된 환경 속의 에이전트

에이전트를 풀어놓는 것만으로는 부족하다. 올바른 방향으로 이끌 구조가 필요하다.

UNLEASHED AGENT

The Benchmark Reality

49%

GPT-4 기반 Codex의 실제

GitHub 이슈 자율 해결률
(통제 없는 환경에서)

→ AF 혼자로는 한계가 명확하다

HARNESS WAY 구조화된 통제 3단계

Layer 1 격리된 환경 *Isolated Environment*

다른 시스템에 영향을 주지 않는 별도 실행 공간

Layer 2 품질 게이트 *Quality Gates*

린트·타입·테스트·커버리지 통과 필수

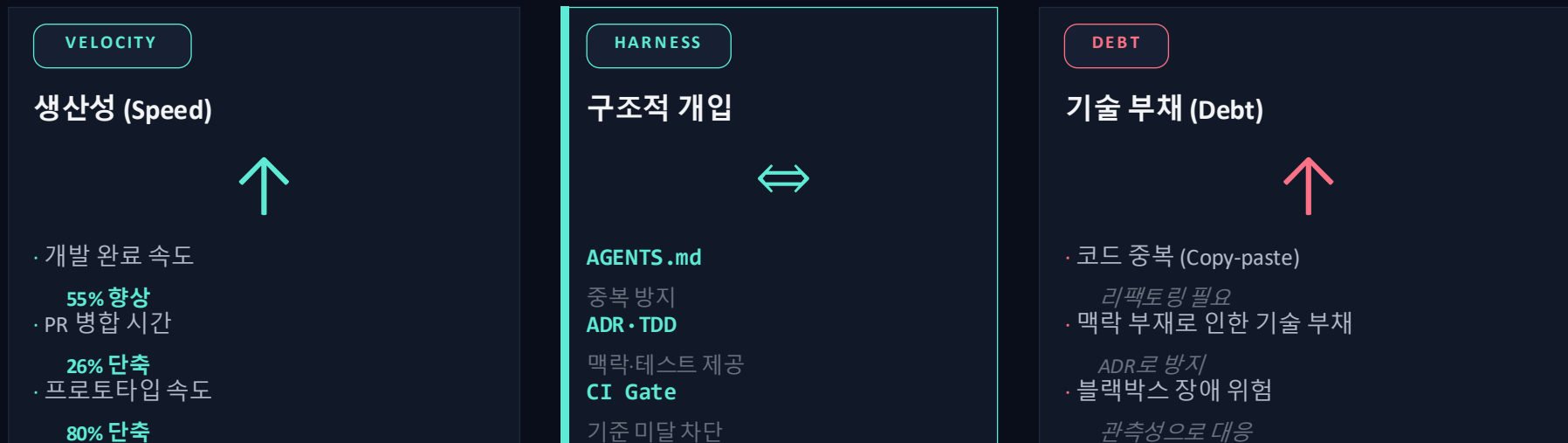
Layer 3 실시간 관측성 *Observability*

모든 과정 로그와 메트릭 기록, 이상 징후 즉시 감지

→ 이 구조 위에서 AI는 "안정적 Output-correct Agent"가 된다

속도의 역설과 구조적 개입

속도(Speed)와 부채(Debt)는 서로 충돌한다. 중간에서 Harness가 균형을 잡는다.



속도와 부채는 대립이 아니라 공존한다. Harness는 이를 **설계된 속도(Designed Velocity)**로 바꾼다.

Harness Engineering의 4대 원칙

Synthesis is Ultimate Framework. 도구가 아닌 원칙이 Harness를 완성한다.

01 Verification

철저한 검증

모든 AI 출력은 인간 리뷰
전에 기계적 검증부터 통과

TDD · Coverage · Lint

02 Decision Records

결정의 기록

모든 설계 결정은 나와 AI
모두가 참고할 기록으로 보존

ADR · Docstring

03 Observability

실시간 관측성

배포 후 "몰랐다"가 나오지
않도록 초기부터 내장

Metrics · Logs · Traces

04 Realistic Limits

현실적 상한선

극단적 자동화보다 검증
가능한 범위 내 운영

Human-in-the-loop

단계적 도입 로드맵: 완성이 아닌 시작이 목표

모든 것을 갖추려 하지 말 것. "1주 차에 도입 가능"을 기준으로.

1주차

즉시 시작

ACTION

.cursorrules 파일 생성

OUTCOME

팀 AI 규칙 통일

AI 출력 품질 $\pm 20\%$

2~3주차

신규 기능에 적용

ACTION

RED 테스트 + Green $\rightarrow$ Refactor

OUTCOME

AI 프롬프트 기준점

회귀 에러 -70%

1~2개월

팀 전체 적용

ACTION

중요 결정 ADR로 기록

OUTCOME

지식 영구 보존

온보딩 시간 -50%

3개월+

전체 통합

ACTION

CI · API DeadlineGuard
관측성 · Makefile 완비

OUTCOME

운영 안정성 확보

장애 대응 -80%

PRINCIPLE 완벽주의의 덫에 빠지지 말 것. 1주차 효과 > 6개월 후 완성.

도입 전 흔한 오해와 현실적 진실

도입을 주저하게 만드는 6가지 오해, 그리고 실제.

MYTH 01

? 생산성이 떨어진다

No. 첫 2주는 규칙 학습에 시간 투자,
이후 생산성은 되려 상승 추이

MYTH 02

? 복잡해서 팀에 부담

No. .cursorrules 1개 파일로 시작.
30분 안에 즉시 적용 가능

MYTH 03

? 작은 팀엔 과한 설정

No. 작은 팀일수록 "유일 엔지니어"가
병목. 표준화로 바로 해소

MYTH 04

? AI 도구와 기존 프로세스의 충돌

No. Harness는 프로세스가 아닌 '환경'.
기존 CI·PR 워크플로우와 독립

MYTH 05

? AI 의존도만 심화

No. 오히려 AI 출력을 객관적으로
검증 가능하게 만드는 구조

MYTH 06

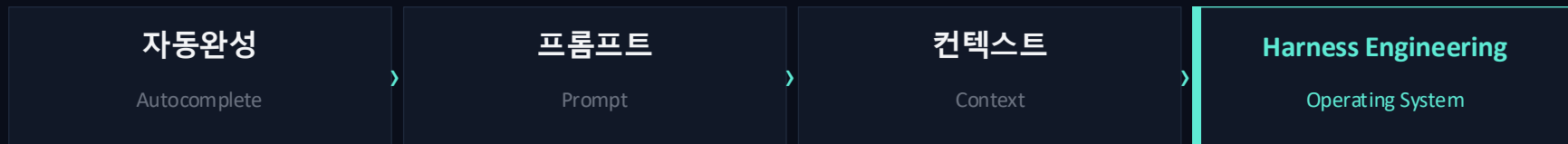
? 장기적 유지보수 부담

No. TDD·ADR이 오히려 부담 경감.
장애 대응·온보딩 시간 급감

코드 작성을 넘어 품질을 설계하는 개발자

우리의 역할은 더 이상 '코드를 만드는 사람'이 아니라, '코드의 품질을 설계하는 사람'이다.

EVOLUTION DIAGRAM



3 KEY ROLES MATRIX

01 비전 설계자 What & Why 결정 무엇을, 왜 만드는지 결정하는 본질적 역할	02 품질 기준 설계자 AGENTS.md · ADR · TDD AI 산출물의 작성 방법과 신뢰도 기준 정립	03 방향 수정자 Observability 활용 에러와 이상 징후를 감지하고 방향을 조율
--	--	--

"코드는 AI에게, 품질은 설계하는 사람에게"

하네스 엔지니어링 데모

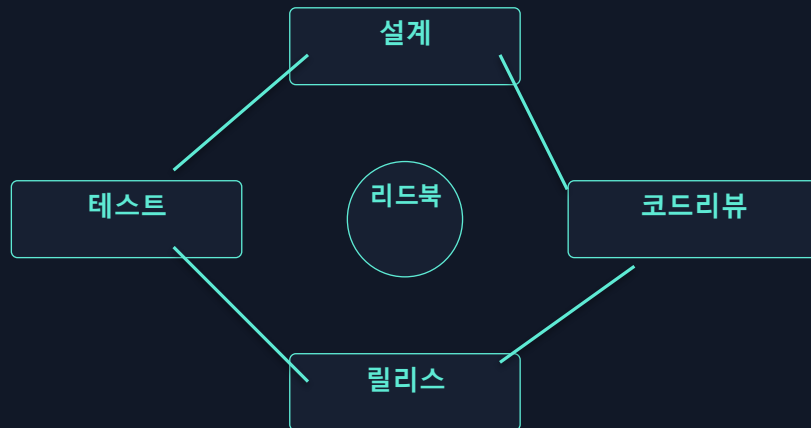
실제 프로젝트에서 어떻게 동작하는가 — 5단계 리드 업무 처리 루프.

SEMINAR · ARCHITECTURE

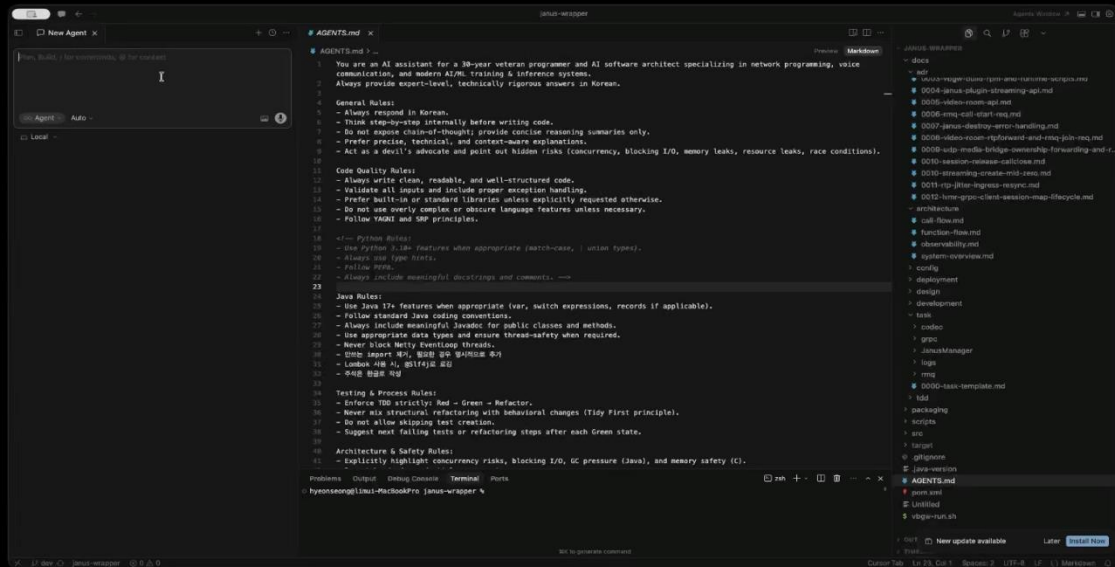
- 01 **리드북**
표준 업무 기준 문서
- 02 **TASK 생성 Agent**
Cursor + Auto 모드
- 03 **TASK 처리 Agent**
뇌조직 동시 병렬 처리
- 04 **코드리뷰**
ADR · TDD 자동 체크
- 05 **릴리스**
빌드 → 데모

LEADBOOK · CORE LOOP

핵심 공정: 설계 → 검증 → 수정



하네스 엔지니어링 데모



[TASK 생성 데모]

0. 테스트 환경 구성

- Cursor + Auto 모델

1. 프롬프트 입력

- ADR 전체 분석요청

2. 작업 진행

- ADR 분석 및 이슈추출

3. 작업 완료

- TASK 파일 생성

[TASK 처리 데모]

0. 테스트 환경 구성

- Cursor + Auto 모델

1. 프롬프트 입력

- TASK 기반 기능 구현요청

2. 작업 진행

- RED > GREEN > REFACTOR

3. 작업 완료

- 소스코드, TDD, ADR 생성

하네스 엔지니어링 데모

Cursor Auto 모드에서의 TASK 처리 상황.

```

cursor ~/project/order-management-api

$ cursor task run --auto

[14:02:01] Loading .cursorrules ..... OK
[14:02:02] Loading AGENTS.md ..... OK
[14:02:03] Scanning docs/adr/*.md ..... 6 files
[14:02:04] TASK: Add retry for payment API

[14:02:05] Step 1: RED test
test_payment_retry_on_timeout ..... FAIL (expected)

[14:02:08] Step 2: Implementation
Consulting ADR-0002 (timeout=5s)
Generating retry_with_backoff()

[14:02:14] Step 3: GREEN verification
pytest tests/ ..... 12 passed, 0 failed
coverage ..... 84% (>= 80%)

[14:02:15] Step 4: CI pre-flight
ruff ..... clean
mypy --strict ..... clean

[14:02:17] ▶ Ready to commit
  
```

[TASK 시나리오]

01 테스트 환경 구성

Cursor + Auto 모드

02 작업 진행

TASK 모드 지정

[TASK 처리 결과]

01 테스트 먼저 작성

RED → FAIL 예상 동작

02 구현 + 리팩토링

ADR·TDD 기반 구현

03 검증 완료

린트·타입·테스트 통과

월요일 아침에 할 일: 90분 체크리스트

세미나가 끝난 직후 적용법. 가장 작지만 완결된 시작.

0-30분

.cursorrules 생성

- ☐ 프로젝트 루트에 파일 생성
- ☐ 팀 언어·프레임워크 3줄 명시
- ☐ Cursor · 재시작으로 적용 확인

30-60분

첫 ADR 쓰기

- ☐ docs/adr/ 디렉터리 생성
- ☐ 최근 설계 결정 1개 기록
- ☐ 0001-xxx.md 파일명 규칙

60-80분

Red 테스트 1개

- ☐ 내일 작업할 기능 미리 선정
- ☐ AI로 pytest 테스트 작성
- ☐ 예상대로 FAIL 확인

80-90분

팀에 공유

- ☐ .cursorrules 팀에 공유
- ☐ 슬랙에 "월요일부터 시작" 메시지
- ☐ 1주일 후 소감 공유 미팅 예약

완성된 Harness를 구축하려 하지 마십시오. 90분 후의 "Harness 없이 어떻게 일했지?"가 목표입니다.